

ภาคผนวก ข.

ชุดคำสั่งควบคุมอุปกรณ์สำหรับหุ่นยนต์และการนำไปใช้

ชุดคำสั่งควบคุมอุปกรณ์สำหรับหุ่นยนต์แบ่งเป็น 2 ส่วน

1. ส่วนของแฟ้มข้อมูลแบบไลเบอรรี่ ที่เก็บคลาสของอุปกรณ์ชนิดต่างไว้
2. ส่วนของทาสก์ที่ใช้ควบคุมอุปกรณ์แต่ละชนิด

■ ส่วนของแฟ้มข้อมูลแบบไลเบอรรี่

จะประกาศคลาสต่างๆไว้ในแฟ้มข้อมูล devices.h และจะเก็บโปรแกรมแสดงการทำงานของเมททอดในแต่คลาสไว้ในแฟ้มข้อมูล devices.cpp เพื่อให้ผู้ใช้งานเรียกใช้ได้ง่าย

แฟ้มข้อมูล devices.h

```
//----- DC MOTOR -----
class DCMotor
{
private :
    OS_EVENT *Mbox;
    INT16U port_out1;
    char port_status;
public :
    char name[12];
    void setDCMotor(OS_EVENT *M,INT16U p1,int p2,char p3,char N[]);
    void forward();
    void backward();
    void stop();
};

//----- STEP MOTOR -----

class STEPMotor
{
private :
    OS_EVENT *Mbox;
    int port_out;
    char port_status;
public :
    char name[12];
    void setSTEPSMotor(OS_EVENT *M,INT16U p1,char p3,char N[]);
    void forward ();
    void backward();
    void stop();
};
```

```
//----- IR SENSOR -----

class IRsensor
{
    private :
        OS_EVENT *Mbox;
        INT16U port_in;
    public :
        char name[12];
        void setIRsensor(OS_EVENT *M,INT16U p1,char N[]);
        int detect();
};

//----- BUMPED SENSOR -----

class BUMPSensor
{
    private :
        OS_EVENT *Mbox;
        INT16U port_in;
    public :
        char name[12];
        void setBUMPSensor(OS_EVENT *M,INT16U p1,char N[]);
        int bump();
};

//----- End all Class of devices.h -----
```

จากแฟ้มข้อมูล devices.h จะแสดงค่าแอดทริบิวและเมทอดของคลาสต่างๆโดยจะเห็นว่าแต่ละคลาสจะประกอบด้วยส่วนหลักๆดังนี้

- แอดทริบิว Mbox เป็นตัวแปรชนิด พอยเตอร์ของ OS_EVENT ซึ่งจะมีไว้สำหรับอ้างถึงเมล์บ็อกที่ใช้ในการสื่อสารระหว่างอุปกรณ์ กับทาสก์ที่ควบคุมอุปกรณ์ชนิดนั้นๆ โดยอุปกรณ์ 1 ตัวจะมี 1 เมล์บ็อก
- แอดทริบิว port_in,port_out เป็นตัวแปรชนิด int ซึ่งจะใช้เก็บว่าเป็นอุปกรณ์ตัวใดในจำนวนอุปกรณ์ที่เป็นชนิดเดียวกันทั้งหมด หรือก็คือการกำหนดหมายเลขให้กับอุปกรณ์เพื่อสามารถสั่งงานอุปกรณ์ได้ตรงตามที่ต้องการ
- แอดทริบิว name เป็นตัวแปรชนิด อาร์เรย์ของ char ซึ่งจะใช้เก็บชื่อของอุปกรณ์ตามที่ผู้ผู้กำหนดไว้
- เมทอด ที่ขึ้นต้นด้วยคำว่า set...(...) จะเป็นเมทอดที่ใช้ในการกำหนดค่าแอดทริบิวต่างๆให้กับคลาส

ส่วนของเมทอดนอกเหนือจากนั้นจะเป็นเมทอดที่ใช้ในการสั่งงานควบคุมอุปกรณ์ของแต่ละ

ชนิด

เพิ่มข้อมูล devices.cpp

```
#include "includes.h"

//----- DC MOTOR -----
void DCmotor ::setDCmotor(OS_EVENT *M,INT16U p1,char p3,char N[])
{
    Mbox = M;
    port_out1 = p1;
    port_status = p3;
    strcpy(name,N);
}
void DCmotor::forward()
{
    static char jms[3];

    port_status = jms[port_out1-1];

    jms[port_out1-1] = 0x01<<((port_out1-1)*2);

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms[port_out1-1]);
}
void DCmotor::backward()
{
    static char jms[3];

    port_status = jms[port_out1-1];

    jms[port_out1-1] = 0x02<<((port_out1-1)*2);

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms[port_out1-1]);
}
void DCmotor::stop()
{
    static char jms;

    port_status = jms;

    jms = 0;
    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms);
}
```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุมดีซีมอเตอร์ โดยทั่วไปจะเป็นการคำนวณค่าของ ดีซีมอเตอร์ แต่ละตัวเพื่อส่งให้ทาสก์ดีซีมอเตอร์ทางเมล์บ็อก ประมวลผลส่งไปควบคุมดีซีมอเตอร์จริงๆ

```
//----- STEP MOTOR -----

void STEPMotor::setSTEPMotor(OS_EVENT *M,INT16U p1,char p3,char N[])
{
    char s[80];
    INT8U err;
    Mbox = M;
    port_out = p1;
    port_status = p3;
    strcpy(name,N);
}

void STEPMotor::forward()
{
    static char jms[2];

    port_status = jms[port_out-1];

    if (port_out==1)
    {
        jms[0] =12;
    }
    else
    {
        jms[1] =3;
    }

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms[port_out-1]);
}

void STEPMotor::backward()
{
    static char jms[2];

    port_status = jms[port_out-1];

    if (port_out==1)
    {
        jms[0] = 8;
    }
    else
    {
        jms[1] =2;
    }

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms[port_out-1]);
}

void STEPMotor::stop()
{
    static char jms;

    port_status = jms;
    jms = 0;

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms);
}
```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุม สเต็ปมอเตอร์ โดยทั่วไปจะเป็นการคำนวณค่าของ สเต็ปมอเตอร์ แต่ละตัวในที่นี้ใช้ควบคุม สเต็ปมอเตอร์ 2 ตัวเพื่อส่งให้ทาสก์สเต็ปมอเตอร์ผ่านทางเมล์บ็อก ประมวลผลส่งไปควบคุมสเต็ปมอเตอร์จริงๆ

//----- IR SENSOR -----

```
void IRsensor::setIRsensor(OS_EVENT *M,INT16U p1,char N[])
{
    Mbox = M;
    port_in = p1;
    strcpy(name,N);
}
int IRsensor::detect()
{
    char *IRin;

    IRin= (char *) OSMboxAccept(Mbox);
    OSMboxPost(Mbox,IRin);

    if( *IRin== 0x10 )
    {
        return 1;
    }
    else return 0;
}
```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุม อินฟราเรดเซ็นเซอร์ จะเป็นการรับข้อมูลมาจากเมล์บ็อกที่ส่งมาจากทาสก์ของ อินฟราเรดเซ็นเซอร์ โดยทาสก์อินฟราเรดเซ็นเซอร์ จะส่งค่ามาบอกว่าอินฟราเรดเซ็นเซอร์ตัวนั้น มีค่าเป็นอะไร(ทำงานอยู่หรือไม่ทำงาน) แล้วเมทอด detect() จะส่งค่ากลับไปให้ผู้เรียกว่า อินฟราเรดเซ็นเซอร์ตัวนั้น ทำงานอยู่หรือไม่ ถ้าทำงานจะส่งค่า 1 ถ้าไม่ทำงานจะส่งค่า 0

//----- BUMPED SENSOR -----

```
void BUMPSensor::setBUMPSensor(OS_EVENT *M,INT16U p1,char N[])
{
    Mbox = M;
    port_in = p1;
    strcpy(name,N);
}
int BUMPSensor::bump()
{
    char *Bump;

    Bump= (char *) OSMboxAccept(Mbox);
    OSMboxPost(Mbox,Bump);
    if( *Bump ==1)
        { return 1;}
    else { return 0;}
}
```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุม บัมเซ็นเซอร์ จะเป็นการรับข้อมูลมาจาก เมลล์บ็อกที่ส่งมาจากทาสก์ของ บัมเซ็นเซอร์ โดยทาสก์ อินฟารเคชันเซอร์ จะส่งค่ามาบอกว่า บัมเซ็นเซอร์ ตัวนั้น มีค่าเป็นอะไร(ถูกกระทบหรือไม่ถูกกระทบ) แล้วเมทอด bump() จะส่งค่ากลับไปให้ผู้เรียกว่า บัมเซ็นเซอร์ตัวนั้น ถูกกระทบอยู่หรือไม่ ถ้าถูกกระทบจะส่งค่า 1 ถ้าไม่ถูกกระทบจะส่งค่า 0

ตัวอย่างวิธีการเรียกใช้ชุดคำสั่งที่เป็นไลบรารี

```
----- ส่วนของการประกาศออบเจ็กต์ -----
BUMPsensor B1;
STEPmotor step1;
STEPmotor step2;

----- ส่วนของการเรียกใช้งานเมทอด -----
if (B1.bump ())
{
    step1.backward ();
    step2.stop ();

    OSTimeDlyHMSM (0, 0, 1, 0);

    step1.forward ();

    OSTimeDlyHMSM (0, 0, 1, 0);

    step1.stop ();
}
else
{
    step1.stop ();
    step2.backward ();

    OSTimeDlyHMSM (0, 0, 1, 0);

    step2.forward ();

    OSTimeDlyHMSM (0, 0, 1, 0);

    step2.stop ();
}
```

* ส่วนการประกาศออบเจ็กต์ โดยปกติโปรแกรมช่วยเหลือจะประกาศให้ ผู้ใช้สามารถเรียกใช้ได้เลย

■ ส่วนของทาสก์ที่ใช้ควบคุมอุปกรณ์

ซึ่งจะแยกเก็บเป็นแฟ้มข้อมูลที่มีชื่อเหมือนอุปกรณ์นั้นๆ และจะถูกเรียกรวมกับแฟ้มข้อมูลหลักเมื่อผู้ใช้เลือกอุปกรณ์ชนิดนั้น โดยจะมีหน้าที่ควบคุมอุปกรณ์นั้นๆ โดยตรง แบ่งเป็น

ทาสก์ดีซีเอ็มโอเตอร์

```
void Task2 (void *data)
{
    char *rxmsg;
    int sh,oldsh;
    data = data;

    for (;;) {
        oldsh=sh;
        rxmsg = (char *) OSMboxAccept (DCMbox);
        sh=*rxmsg;

        if ((sh != 116) &&( sh!= oldsh))
        {
            outportb(0x3020,sh);
        }

        OSTimeDlyHMSM (0, 0, 1, 0);           // Wait 1 second
    }
}
```

ทาสก์ดีซีเอ็มโอเตอร์จะรับข้อมูลมาจากเมล็บบ็อกที่ชื่อ DCMbox แล้วนำมาตรวจสอบค่าก่อนส่งข้อมูลออกไปทาง ไอช่าบัสตำแหน่งที่ 3020 เพื่อควบคุมดีซีเอ็มโอเตอร์ จากโปรแกรมข้างบนยังควบคุมดีซีเอ็มโอเตอร์ 1 ตัวหากต้องการควบคุมเพิ่มจะต้องเพิ่ม เมล็บบ็อกที่ส่งค่ามาให้สำหรับควบคุมดีซีเอ็มโอเตอร์ และเพิ่มส่วนประมวลผลจัดการสำหรับส่งข้อมูลออกไอช่าบัส โดยไอช่าบัส 1 ตำแหน่งควบคุมดีซีเอ็มโอเตอร์ได้สูงสุด 4 ตัว ถ้าต้องการควบคุมดีซีเอ็มโอเตอร์มากกว่านั้นต้องเพิ่มการอ้างตำแหน่งของไอช่าบัส ส่วนของการหน่วงเวลาช่วงท้ายเป็นการกระทำเพื่อให้ทาสก์อื่นๆ สามารถแย่งชิงตัวประมวลผลไปทำงานได้ทำให้ไม่มีทาสก์ใดควบคุมตัวประมวลผลเพียงตัวเดียว

ทาสก์สเต็ปโมเตอร์

```
void Task3 (void *data)
{
    char *step1;
    char *step2;
    int sh,oldsh,sh1,sh2;

    sh=0;
    *step1=0;
    *step2=0;
    data = data;

    for (;;) {
        sh1=*step1;
        sh2=*step2;
        oldsh=sh;

        step1 = (char *) OSMboxAccept(Step1Mbox);
        OSMboxPost(Step1Mbox,step1);
    }
}
```

```

step2 = (char *)OSMboxAccept(Step2Mbox);
OSMboxPost(Step2Mbox,step2);

if (*step1==116)
    { *step1=sh1;}
if (*step2==116)
    { *step2=sh2;}
sh=*step1|*step2|0x10;

if ((sh != 116) &&( sh!= oldsh))
    { outportb(0x3001,sh); }
    OSTimeDlyHMSM(0, 0, 0,70);
}
}

```

ทาสก์เตปมอเตอร์จะรับข้อมูลมาจากเมล์บ็อกที่ชื่อ Step1Mbox และ Step2Mbox สำหรับควบคุมสแตปมอเตอร์ 2 ตัว แล้วนำมาประมวลผลรวมค่าที่ได้จากเมล์บ็อกทั้งสอง ส่งข้อมูลออกไปทางไอซีบัสดำแหน่งที่ 3001 เพื่อควบคุมสแตปมอเตอร์ จากโปรแกรมข้างบนยังควบคุมสแตปมอเตอร์ 2 ตัว หากต้องการควบคุมเพิ่มจะต้องเพิ่ม เมล์บ็อกที่ส่งค่ามาให้สำหรับควบคุมสแตปมอเตอร์และเพิ่มส่วนประมวลผลจัดการสำหรับส่งข้อมูลออกไอซีบัสด โดยไอซีบัสด 1 ตำแหน่งควบคุมสแตปมอเตอร์ได้สูงสุด 3 ตัว (สามารถดัดแปลงให้ควบคุมถึง 4 ตัวได้) ถ้าต้องการควบคุมสแตปมอเตอร์มากกว่านั้นต้องเพิ่มการอ้างตำแหน่งของไอซีบัสด

ทาสก์อินฟราเรดเซ็นเซอร์

```

void Task4 (void *data)
{
    INT16U pt;
    char inIR,inIR1;
    char true,false;

    data = data;
    true=0x10;
    false=0x11;

    for (;;) {
        pt=0x3004;
        inIR = inportb(pt);

        inIR1=inIR&(0x01<<0);
        if (inIR1==0x01)
        {
            OSMboxAccept(IR1Mbox);
            OSMboxPost(IR1Mbox, (void *)true);
        }
        else
        {
            OSMboxAccept(IR1Mbox);
            OSMboxPost(IR1Mbox, (void *)false);
        }
    }
}

```



```

inIR1=inIR&(0x01<<1);
if (inIR1==0x02)
{
    OSMboxAccept(IR2Mbox);
    OSMboxPost(IR2Mbox, (void *)true);
}
else
{
    OSMboxAccept(IR2Mbox);
    OSMboxPost(IR2Mbox, (void *)false);
}

inIR1=inIR&(0x01<<2);
if (inIR1==0x04)
{
    OSMboxAccept(IR3Mbox);
    OSMboxPost(IR3Mbox, (void *)true);
}
else
{
    OSMboxAccept(IR3Mbox);
    OSMboxPost(IR3Mbox, (void *)false);
}

inIR1=inIR&(0x01<<3);
if (inIR1==0x08)
{
    OSMboxAccept(IR4Mbox);
    OSMboxPost(IR4Mbox, (void *)true);
}
else
{
    OSMboxAccept(IR4Mbox);
    OSMboxPost(IR4Mbox, (void *)false);
}

inIR1=inIR&(0x01<<4);
if (inIR1==0x10)
{
    OSMboxAccept(IR5Mbox);
    OSMboxPost(IR5Mbox, (void *)true);
}
else
{
    OSMboxAccept(IR5Mbox);
    OSMboxPost(IR5Mbox, (void *)false);
}

inIR1=inIR&(0x01<<5);
if (inIR1==0x20)
{
    OSMboxAccept(IR6Mbox);
    OSMboxPost(IR6Mbox, (void *)true);
}
else
{
    OSMboxAccept(IR6Mbox);
    OSMboxPost(IR6Mbox, (void *)false);
}

```

```

        inIR1=inIR&(0x01<<6);
        if (inIR1==0x40)
        {
            OSMboxAccept(IR7Mbox);
            OSMboxPost(IR7Mbox, (void *)true);
        }
        else
        {
            OSMboxAccept(IR7Mbox);
            OSMboxPost(IR7Mbox, (void *)false);
        }

        inIR1=inIR&(0x01<<7);
        if (inIR1==0x80)
        {
            OSMboxAccept(IR8Mbox);
            OSMboxPost(IR8Mbox, (void *)true);
        }
        else
        {
            OSMboxAccept(IR8Mbox);
            OSMboxPost(IR8Mbox, (void *)false);
        }
        OSTimeDlyHMSM(0, 0, 0,30);
    }
}

```

ทาสก์อินฟารเคชั่นเซอร์จะรับข้อมูลมาทางตำแหน่งที่ 3004 ของไอช่าบัสแล้วนำมาประมวลผลว่าอินฟารเคชั่นเซอร์แต่ละตัวมีค่าเป็นอะไรแล้วส่งค่าไปเก็บไว้ในเมล์บ็อกของอินฟารเคชั่นเซอร์แต่ละตัว เพื่อให้ผู้ใช้งานสามารถเรียกค่าของอินฟารเคชั่นเซอร์แต่ละตัวมาใช้งานได้ทันที โดยโปรแกรมข้างบนจะจัดการกับอินฟารเคชั่นเซอร์จำนวน 8 ตัว ถ้าใช้น้อยกว่าจำนวนที่กำหนดให้สามารถแก้ไขกลับส่วนที่ไม่ใช่ออกไปได้ แต่ถ้าต้องการเพิ่มจำนวนต้องมีการเพิ่มการอ้างตำแหน่งของไอช่าบัสในการรับค่าเนื่องจาก ไอช่าบัส 1 ตำแหน่งควบคุมอินฟารเคชั่นเซอร์ได้จำนวน 8 ตัว

ทาสก์บัมเซ็นเซอร์

```

void Task5 (void *data)
{
    char true,false;
    INT8U inBump,tempbump,tem;

    data = data;
    true=0x01;
    false=0x11;

```

```

for (;;) {
    inBump = inportb(0x3010);

    tempbump=inBump|0xfe;
    if (tempbump==0xfe)
    {
        OSMboxAccept(Bump1Mbox);
        OSMboxPost(Bump1Mbox, (void *)&true);
    }
    else
    {
        OSMboxAccept(Bump1Mbox);
        OSMboxPost(Bump1Mbox, (void *)&false);
    }

    tempbump=inBump|0xfd;
    if (tempbump ==0xfd)
    {
        OSMboxAccept(Bump2Mbox);
        OSMboxPost(Bump2Mbox, (void *)&true);
    }
    else
    {
        OSMboxAccept(Bump2Mbox);
        OSMboxPost(Bump2Mbox, (void *)&false);
    }

    tempbump=inBump|0xfb;
    if (tempbump==0xfb)
    {
        OSMboxAccept(Bump3Mbox);
        OSMboxPost(Bump3Mbox, (void *)&true);
    }
    else
    {
        OSMboxAccept(Bump3Mbox);
        OSMboxPost(Bump3Mbox, (void *)&false);
    }

    tempbump=inBump|0xf7;
    if (tempbump==0xf7)
    {
        OSMboxAccept(Bump4Mbox);
        OSMboxPost(Bump4Mbox, (void *)&true);
    }
    else
    {
        OSMboxAccept(Bump4Mbox);
        OSMboxPost(Bump4Mbox, (void *)&false);
    }
}

```

```

tempbump=inBump|0xef;
if (tempbump==0xef)
{
    OSMboxAccept(Bump5Mbox);
    OSMboxPost(Bump5Mbox, (void *)&true);
}
else
{
    OSMboxAccept(Bump5Mbox);
    OSMboxPost(Bump5Mbox, (void *)&false);
}

tempbump=inBump|0xdf;
if (tempbump==0xdf)
{ tem=1;
    OSMboxAccept(Bump6Mbox);
    OSMboxPost(Bump6Mbox, (void *)&true);
}
else
{
    OSMboxAccept(Bump6Mbox);
    OSMboxPost(Bump6Mbox, (void *)&false);
}

tempbump=inBump|0xbf;
if (tempbump==0xbf)
{
    OSMboxAccept(Bump7Mbox);
    OSMboxPost(Bump7Mbox, (void *)&true);
}
else
{
    OSMboxAccept(Bump7Mbox);
    OSMboxPost(Bump7Mbox, (void *)&false);
}

tempbump=inBump|0x7f;
if (tempbump==0x7f)
{
    OSMboxAccept(Bump8Mbox);
    OSMboxPost(Bump8Mbox, (void *)&true);
}
else
{
    OSMboxAccept(Bump8Mbox);
    OSMboxPost(Bump8Mbox, (void *)&false);
}

    OSTimeDlyHMSM(0, 0,1,0);
}
}

```

ทาสก์บัมเซ็นเซอร์จะทำงานคล้ายกับ ทาสก์อินฟาเรดเซ็นเซอร์ แต่ต่างกันที่ ทาสก์บัมเซ็นเซอร์ทำงานที่ค่าเป็น 0 และค่าปกติเป็น 1

โดยทาสก์ต่างจะถูกรวมไว้ภายใต้โครงสร้างการทำงานของไมโครซีไอเอส2 ซึ่งมีรูปแบบดังนี้

```
#include "includes.h"

#define TASK_STK_SIZE 512          /// กำหนด ขนาด สแต็ก ของทาสก์

///////////////////////////////// กำหนด ID ///////////////////////////////////
#define TASK_START_ID 0           /* Application tasks IDs          */
#define TASK_CLK_ID 1
#define TASK_1_ID 2
#define TASK_2_ID 3
#define TASK_3_ID 4
#define TASK_4_ID 5
#define TASK_5_ID 6

///////////////////////////////// กำหนด Priority ///////////////////////////////////
#define TASK_START_PRIO 10        /* Application tasks priorities      */
#define TASK_CLK_PRIO 17
#define TASK_1_PRIO 12
#define TASK_2_PRIO 13
#define TASK_3_PRIO 14
#define TASK_4_PRIO 15
#define TASK_5_PRIO 16

///////////////////////////////// กำหนด task stack ///////////////////////////////////
OS_STK TaskStartStk[TASK_STK_SIZE]; /* Startup task stack              */
OS_STK TaskClkStk[TASK_STK_SIZE];   /* Clock task stack                 */
OS_STK Task1Stk[TASK_STK_SIZE];     /* Task #1 task stack               */
OS_STK Task2Stk[TASK_STK_SIZE];     /* Task #2 task stack               */
OS_STK Task3Stk[TASK_STK_SIZE];     /* Task #3 task stack               */
OS_STK Task4Stk[TASK_STK_SIZE];     /* Task #4 task stack               */
OS_STK Task5Stk[TASK_STK_SIZE];     /* Task #5 task stack               */

///////////////////////////////// กำหนด task ///////////////////////////////////
void TaskStart(void *data);          /* Function prototypes of tasks    */
static void TaskStartCreateTasks(void);
static void TaskStartDisplnit(void);
static void TaskStartDisplnit2(int);
static void TaskStartDisp(void);
        void TaskClk(void *data);
        void Task1(void *data);
        void Task2(void *data);
        void Task3(void *data);
        void Task4(void *data);
        void Task5(void *data);

OS_EVENT *DispSem  /// SEMAPHORES สำหรับแสดงผล ///
/// create os event for mailbox for device //////////////////////////////////

OS_EVENT *DCMbox;
OS_EVENT *Step1Mbox;
OS_EVENT *Step2Mbox;
OS_EVENT *IR1Mbox;
OS_EVENT *IR2Mbox;
OS_EVENT *IR3Mbox;
OS_EVENT *Bump1Mbox;
OS_EVENT *Bump2Mbox;
OS_EVENT *Bump3Mbox;
```

```

OS_EVENT *IR4Mbox;
OS_EVENT *IR5Mbox;
OS_EVENT *IR6Mbox;
OS_EVENT *IR7Mbox;
OS_EVENT *IR8Mbox;

OS_EVENT *Bump4Mbox;
OS_EVENT *Bump5Mbox;
OS_EVENT *Bump6Mbox;
OS_EVENT *Bump7Mbox;
OS_EVENT *Bump8Mbox;
OS_EVENT *SonarMbox;
OS_EVENT *Senddis1;
OS_EVENT *Senddis2;
//////////////////// จม create os event for mailbox for device////////////////////

//////////////////// ประกาศ object ตามชื่อ : ชื่อคลาส ชื่อออบเจกต์(ที่กำหนด) //////////////////////

//// ตัวอย่าง ////
BUMPsensor B1;
BUMPsensor B2;
STEPmotor step1;
STEPmotor step2;

//////////////////// จม ประกาศ object////////////////////

void main (void) ////////////////// main uCOS //////////////////
{
    OS_STK *ptos;
    OS_STK *pbos;
    INT32U size;

    COM86_DisplnIt(); //กำหนดค่าเริ่มต้นของหน้าจอแสดงผล////
    COM86_DisplClrScr(DISP_BLACK, DISP_WHITE);          /* Clear the screen */

    OSInit();                                           /* Initialize uC/OS-II */
    COM86_DOSSaveReturn();                             /* Save environment to return to DOS */
    COM86_VectSet(uCOS, OSCtxSw);                      /* Install uC/OS-II's context switch */
    vector */
    DispSem = OSSemCreate(1);                          /* Console display semaphore */
    COM86_ElapsedInit();                               /* Initialized elapsed time measurement */

    ptos = &TaskStartStk[TASK_STK_SIZE - 1];          /* TaskStart() will use Floating-
Point */
    pbos = &TaskStartStk[0];
    size = TASK_STK_SIZE;

    OSTaskStkInit_FPE_x86(&ptos, &pbos, &size);

    ,

    ////////////////// สร้าง taskstart //////////////////

    OSTaskCreateExt(TaskStart,
        (void *)0,
        ptos,
        TASK_START_PRIO,
        TASK_START_ID,
        pbos,

```

```

        size,
        (void *)0,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
OSStart(); /* Start multitasking */
////////// ฉบับ สร้าง taskstart //////////
}
/*
*****
*
*                      STARTUP TASK
*****
*/

void TaskStart (void *pdata)
{
    #if OS_CRITICAL_METHOD == 3 /* Allocate storage for CPU status
    register */
        OS_CPU_SR cpu_sr;
    #endif
    char s[80];
        INT16S key;
        INT8U err;
        pdata = pdata; /* Prevent compiler warning */

    TaskStartDispInit(); /* Setup the display */

    OS_ENTER_CRITICAL(); /* Install uC/OS-II's clock tick ISR */
    COM86_VectSet(0x08, OSTickISR);
    COM86_SetTickRate(OS_TICKS_PER_SEC); /* Reprogram tick rate */
    OS_EXIT_CRITICAL();

    OSStatInit(); /* Initialize uC/OS-II's statistics */

    //////////// create mail box ////////////
    /* Create message mailboxes */

    DCMbox = OSMboxCreate((void *)0);
    Step1Mbox = OSMboxCreate((void *)0);
    Step2Mbox = OSMboxCreate((void *)0);
    IR1Mbox = OSMboxCreate((void *)0);
    IR2Mbox = OSMboxCreate((void *)0);
    IR3Mbox = OSMboxCreate((void *)0);
    Bump1Mbox = OSMboxCreate((void *)0);
    Bump2Mbox = OSMboxCreate((void *)0);
    Bump3Mbox = OSMboxCreate((void *)0);

    IR4Mbox = OSMboxCreate((void *)0);
    IR5Mbox = OSMboxCreate((void *)0);
    IR6Mbox = OSMboxCreate((void *)0);
    IR7Mbox = OSMboxCreate((void *)0);
    IR8Mbox = OSMboxCreate((void *)0);
    Bump4Mbox = OSMboxCreate((void *)0);
    Bump5Mbox = OSMboxCreate((void *)0);
    Bump6Mbox = OSMboxCreate((void *)0);
    Bump7Mbox = OSMboxCreate((void *)0);
    Bump8Mbox = OSMboxCreate((void *)0);

    Senddis1= OSMboxCreate((void *)0);

```

```

Senddis2= OSMboxCreate((void *)0);
SonarMbox = OSMboxCreate((void *)0);

////////// call init variable //////////
////////// เรียกเมททอดในการ กำหนดค่าเริ่มต้นของ แอตทริบิว //////////
step1.setSTEPmotor(Step1Mbox,1,0,"jo");
step2.setSTEPmotor(Step2Mbox,2,0,"jojo");
B1.setBUMPsensor(Bump1Mbox,1,"jo1");
B2.setBUMPsensor(Bump2Mbox,2,"jo2");
//////////จบ กำหนดค่าเริ่มต้น //////////

TaskStartCreateTasks();                /* Create all other tasks          */
    for (;;) {
        TaskStartDisp();                /* Update the display              */
        if (COM86_GetKey(&key)) {        /* See if key has been pressed     */
            if (key == 0x1B) {            /* Yes, see if it's the ESCAPE key */
                COM86_DOSReturn();        /* Yes, return to DOS              */
            }
        }
        OSCtxSwCtr = 0;                  /* Clear context switch counter    */
        OSTimeDly(OS_TICKS_PER_SEC);     /* Wait one second                 */
    }
}
////////// จบ task main uCOS-
//////////

////////// task กำหนดค่าเริ่มต้นในการแสดงผล //////////
static void TaskStartDispInit (void)
{
    COM86_DispStr( 1, 1, "                Robot Mornitor
", DISP_WHITE + DISP_BLINK, DISP_RED);
    COM86_DispStr( 1, 2, "
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 3, " Device
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 4, " Stepmotor
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 5, " Stepmotor
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 6, " IRsensor    (IR8-IR0)
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 7, " Bumpsensor    (bump8-bump0)
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 23, "                <-PRESS 'ESC' TO QUIT->
", DISP_BLACK + DISP_BLINK, DISP_WHITE);
}
////////// จบ task กำหนดค่าเริ่มต้นในการแสดงผล //////////
////////// task กำหนดค่าการแสดงผล //////////
static void TaskStartDisp (void)
{
    char s[80];
    INT8U err;
    char *temp;
    char temp1;

    temp = (char *)OSMboxAccept(Step1Mbox);
    OSMboxPost(Step1Mbox, temp);

    if (*temp ==12)

```



```

{
    sprintf(s, "%s\tforward ", step1.name);
    OSSemPend(DispSem, 0, &err);
    COM86_DispatchStr(25, 4, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

else if (*temp == 8)
{
    sprintf(s, "%s\tbackward", step1.name);
    OSSemPend(DispSem, 0, &err);
    COM86_DispatchStr(25, 4, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

else if (*temp == 0)
{
    sprintf(s, "%s\tstop ", step1.name);
    OSSemPend(DispSem, 0, &err);
    COM86_DispatchStr(25, 4, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

temp = (char *)OSMboxAccept(Step2Mbox);
OSMboxPost(Step2Mbox, temp);

if (*temp == 3)
{
    sprintf(s, "%s\tforward ", step2.name);
    OSSemPend(DispSem, 0, &err);
    COM86_DispatchStr(25, 5, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

else if (*temp == 2)
{
    sprintf(s, "%s\tbackward", step2.name);
    OSSemPend(DispSem, 0, &err);
    COM86_DispatchStr(25, 5, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

else if (*temp == 0)
{
    sprintf(s, "%s\tstop ", step2.name);
    OSSemPend(DispSem, 0, &err);
    COM86_DispatchStr(25, 5, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

temp1=inportb(0x3004);
sprintf(s, "\t%x", temp1);
OSSemPend(DispSem, 0, &err);
COM86_DispatchStr(25, 6, s, DISP_YELLOW, DISP_BLUE);
OSSemPost(DispSem);

sprintf(s, " \t ");
OSSemPend(DispSem, 0, &err);
COM86_DispatchStr(27, 6, s, DISP_YELLOW, DISP_BLUE);
OSSemPost(DispSem);*/

temp1=inportb(0x3010);
sprintf(s, "\t%x ", temp1);
OSSemPend(DispSem, 0, &err);
COM86_DispatchStr(25, 7, s, DISP_YELLOW, DISP_BLUE);

```

```

OSSemPost(DispSem);

    sprintf(s, "%t ");
    OSSemPend(DispSem, 0, &err);
    COM86_DispStr(27, 7, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}
////////// จบ task กำหนดค่าการแสดงผล //////////
/*
*****
*
*          CREATE TASKS
*****
*/
////////// มีไว้สร้าง task //////////
static void TaskStartCreateTasks (void)
{
    //////////// สร้างไว้ตามจำนวนที่ ประกาศไว้ข้างบน
    ////////////
    OSTaskCreateExt(TaskClk, /// ชื่อ //////////
        (void *)0, ///////////ตามมัน//////////
        &TaskClkStk[TASK_STK_SIZE - 1], ////////// stack ตาม top //////////
        TASK_CLK_PRIO, ////////// task prio //////////
        TASK_CLK_ID, ///////////task id //////////
        &TaskClkStk[0], ////////// stack ตาม bottum //////////
        TASK_STK_SIZE, ////////// fix //////////
        (void *)0, ////////// fix //////////
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR); ////////// fix
    ////////////

    OSTaskCreateExt(Task1,
        (void *)0,
        &Task1Stk[TASK_STK_SIZE - 1],
        TASK_1_PRIO,
        TASK_1_ID,
        &Task1Stk[0],
        TASK_STK_SIZE,
        (void *)0,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    OSTaskCreateExt(Task2,
        (void *)0,
        &Task2Stk[TASK_STK_SIZE - 1],
        TASK_2_PRIO,
        TASK_2_ID,
        &Task2Stk[0],
        TASK_STK_SIZE,
        (void *)0,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    OSTaskCreateExt(Task3,
        (void *)0,
        &Task3Stk[TASK_STK_SIZE - 1],
        TASK_3_PRIO,
        TASK_3_ID,
        &Task3Stk[0],
        TASK_STK_SIZE,
        (void *)0,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    OSTaskCreateExt(Task4,

```

```

        (void *)0,
        &Task4Stk[TASK_STK_SIZE-1],
        TASK_4_PRIO,
        TASK_4_ID,
        &Task4Stk[0],
        TASK_STK_SIZE,
        (void *)0,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

OSTaskCreateExt(Task5,
        (void *)0,
        &Task5Stk[TASK_STK_SIZE-1],
        TASK_5_PRIO,
        TASK_5_ID,
        &Task5Stk[0],
        TASK_STK_SIZE,
        (void *)0,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
}
//////////จบ มีไว้สร้าง task //////////
////////// main user //////////
void Task1 (void *pdata)
{
    //////////// ผู้ใช้กำหนดตัวแปล //////////

    pdata=pdata;

    //////////// ผู้ใช้กำหนด โปรแกรม //////////
    for (;;)
    {

    }
}
////////// จบ main user //////////

////////// task ต่างๆที่กล่าวมาข้างต้น //////////
ทาสก์ดีชีมเตอร์
ทาสก์สเติปมอเตอร์
ทาสก์อินฟราเรดเซ็นเซอร์
ทาสก์บีมเซ็นเซอร์
////////// จบ task ต่างๆที่กล่าวมาข้างต้น //////////

***** จบโครงสร้าง uCOS-II *****

```